



功耗分析工具介绍

文档版本
发布日期

V1.3
2025-03-27

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、**UNISOC**、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

紫光展锐（上海）科技有限公司



前言

概述

本文档主要介绍 CPU、GPU、DDR 等常用功耗分析工具，内容涵盖工具的功能、使用方法及运行结果说明，并提供部分有相应功能的命令，帮忙开发人员快速定位问题。

读者对象

文档适用于需要分析功耗问题的开发人员。

符号约定

在本文中可能出现下列标志，它所代表的含义如下。

符号	说明
 说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
 注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
 警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

变更信息

文档版本	发布日期	修改说明
V1.3	2025-03-27	2.1.1 cpu_loading 补充 Kernel6.6 平台获取 CPU loading 信息方式。
V1.2	2024-08-15	- 新增 2.1.4 lit_cpu_freq、2.4.1 loading、2.4.3 top。 - 更新 2.1.3 fix_cpu_freq 工具名称。 - 补充固定 CPU、GPU、DDR 频点的命令。 - 补充部分工具运行结果示例。 - 调整文档结构，优化文档描述。

文档版本	发布日期	修改说明
V1.1	2020-09-03	- 调整文档结构、分类描述内容。 - 补充部分工具使用方法，添加输出结果说明。 - 文档名称由《Unisoc_Power_Tools_Introduction》修改为《功耗分析工具介绍》。
V1.0	2019-10-15	第一次正式发布。

关键字

功耗、频点、GPU、CPU、DDR、使用率。

目 录

1 概述.....	1
2 使用方法.....	2
2.1 CPU	2
2.1.1 cpu_loading	2
2.1.2 cpu_trans_table.....	4
2.1.3 fix_cpu_freq	5
2.1.4 lit_cpu_freq	6
2.2 GPU	7
2.2.1 gpu_loading.....	7
2.2.2 gpu_trans_table	8
2.2.3 fix_gpu_freq	9
2.3 DDR.....	10
2.3.1 ddr_loading.....	10
2.3.2 ddr_trans_table	11
2.3.3 fix_ddr_freq.....	12
2.3.4 ddr_bm	13
2.4 其他	14
2.4.1 loading.....	14
2.4.2 brightness	15
2.4.3 top.....	16

图目录

图 2-1 SC9863A 项目 cpu_loading 运行结果.....	3
图 2-2 UIS7865 项目 cpu_loading 运行结果	3
图 2-3 UMS9632 项目 cpu_loading 运行结果.....	4
图 2-4 SC9863A 项目 gpu_loading 运行结果.....	8
图 2-5 UIS7865 项目 gpu_loading 运行结果.....	8
图 2-6 SC9863A 项目 ddr_loading 运行结果.....	11
图 2-7 UIS7865 项目 ddr_loading 运行结果.....	11
图 2-8 SC9863A 项目 ddr_bm 运行结果	13
图 2-9 UIS7865 项目 ddr_bm 运行结果	14

1 概述

展锐常用的功耗分析工具（脚本程序）集成在 vendor/bin/power 目录下，以便进行问题分析。

功耗分析工具及功能详见表 1-1。

注意

本文档提供的工具仅供参考，如有差异，以实际项目支持情况为准。

表1-1 常用功耗分析工具及功能

大类	细分	功能
CPU	cpu_loading	统计一段时间内 CPU 大小核各个频点的占比和使用情况
	cpu_trans_table	扫描一段时间内 CPU 各个频点的跳变关系
	fix_cpu_freq	固定 CPU 大小核频点
	lit_cpu_freq	限制 CPU 大小核最大频点
GPU	gpu_loading	统计一段时间内 GPU 各个频点的占比和使用情况
	gpu_trans_table	扫描一段时间内 GPU 各个频点的跳变关系
	fix_gpu_freq	固定 GPU 频点
DDR	ddr_loading	统计一段时间内 DDR 各个频点运行时间的占比
	ddr_trans_table	扫描一段时间内 DDR 各个频点的跳变关系
	fix_ddr_freq	固定 DDR 频点
	ddr_bm	抓取一段时间内 DDR 的带宽信息。
其他	loading	同时统计一段时间内 CPU、GPU、DDR 的 loading 信息
	brightness	查看背光节点信息
	top	实时抓取系统中各个进程的资源占用状况

2 使用方法

本章介绍功耗分析工具（脚本程序）的使用方法，并对输出结果中需重点关注的信息进行说明。

注意

本文档提供的工具运行结果仅供参考，如有差异，以实际项目展示为准。

2.1 CPU

2.1.1 cpu_loading

功能

统计一段时间内 CPU 大小核各个频点的占比和使用情况。

cpu_loading 输出结果为二维矩阵，展示了 CPU 大小核的每个频点的闲忙程度。

工具运行时间按需设定，单位：s（秒）。

使用方法

- user root/debug 版本

```
adb shell
cd vendor/bin/power/cpu/loading
./cpu_loading -t 30
```

- user 版本

```
adb shell cp /vendor/bin/power/cpu/loading/cpu_loading data/local/tmp/
adb shell ./data/local/tmp/cpu_loading -t 30
```

-t 后为运行时间，上述命令中 30 代表运行 30s。

运行结果

cpu_loading 运行 30s 后，示例如下。

图2-1 SC9863A 项目 cpu_loading 运行结果

accumulate_time is 60(s)

	768000	884000	1000000	1100000	1200000	online	offline
cpu0	9.92%	0.16%	14.72%	15.18%	45.98%	85.36%	14.64%
cpu1	9.07%	0.14%	13.47%	13.88%	41.51%	78.07%	21.93%
cpu2	7.25%	0.11%	10.77%	11.11%	33.20%	62.45%	37.55%
cpu3	4.69%	0.07%	6.97%	7.18%	21.47%	40.39%	59.61%
freq	11.62%	0.18%	17.25%	17.78%	53.17%		
sum	2920860.38kHz						

小核频率

	768000	1050000	1225000	1400000	1500000	1600000	online	offline
cpu4	0.00%	0.00%	0.00%	0.00%	0.00%	48.11%	48.11%	51.89%
cpu5	0.00%	0.00%	0.00%	0.00%	0.00%	0.45%	0.45%	99.55%
cpu6	0.00%	0.00%	0.00%	0.00%	0.00%	55.86%	55.86%	44.14%
cpu7	0.00%	0.00%	0.00%	0.00%	0.00%	0.03%	0.03%	99.97%
freq	0.00%	0.00%	0.00%	0.00%	0.00%	100.00%		
sum	1671152.25kHz							

大核频率

SUM: 4592012.63kHz

图2-2 UIS7865 项目 cpu_loading 运行结果

testing 30s

小核频点

cpuN	614400	768000	884000	1040000	1228800	1378000	1536000	1612000	online	idle
cpu0	0.60%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.60%	99.40%
cpu1	0.17%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.17%	99.83%
cpu2	0.03%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.03%	99.97%
cpu3	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	100.00%
cpu4	0.03%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.03%	99.97%
cpu5	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	100.00%
Freq	99.87%	0.07%	0.03%	0.03%	0.00%	0.00%	0.03%	0.00%		
sum:	5129.10kHz									
power:	0.00									

大核频点

cpuN	768000	1040000	1228800	1404000	1536000	1612000	online	idle
cpu6	0.07%	0.00%	0.00%	0.00%	0.00%	0.00%	0.07%	99.93%
cpu7	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	100.00%
Freq	100.00%	0.00%	0.00%	0.00%	0.00%	0.00%		
sum:	512.00kHz							
power:	0.00							

sum_total: 5641.10KHZ

power total: 0.00

CPU loading 计算公式: $\text{CPU loading} = \text{Freq} \times \text{online}$

其中:

- cpuN: CPU 大小核的分组情况。
示例中, cpu0~cpu5 属于小核; cpu6、cpu7 属于大核, 具体分组以项目实际情况为准。
- online: 未拔核时 (处于性能状态), 每个 CPU 运行时间对应总时间的占比。
- offline/idle: 非 online 状态 (电源状态), 每个 CPU 运行时间对应总时间的占比。
- Freq: 频点利用率。
- sum_total: 大小核对应 sum 的总和, 和功耗正相关。

- **power_total**: 总负载量, 该值是根据 CPU 大小核各频点的 **power** 值计算得出。其中, 0 表示未配置计算 **power** 值的功能。

除此之外, 搭配 kernel6.6 内核版本的平台可以使用读写节点的方式来获取 **cpuload** 信息, 命令如下。

1. **proc/cpuload_enable** 节点

```
echo x > proc/cpuload_enable
```

- **echo 1**: 注册并开始统计 **cpuload**。
- **echo 0**: 关闭 **cpuload** 接口。

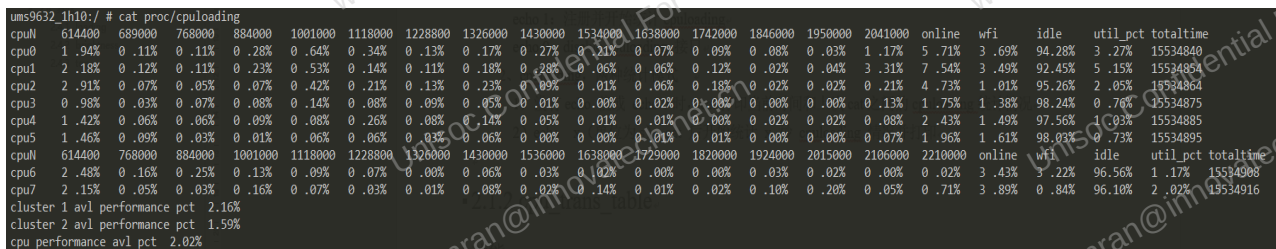
2. **proc/cpuload** 节点

```
echo x > proc/cpuload
```

```
cat proc/cpuload
```

- 无 **echo** 值或 **echo 0** 时, 执行 **cat** 命令后, 打印当前时间和上次 **cat** 命令之间的 **cpuload** 信息。
- **echo x** ($x \geq 1$, 单位: s), 执行 **cat** 命令后, 打印 x 秒后 **cpuload** 信息。

图2-3 UMS9632 项目 **cpu_load** 运行结果



ums9632_1h10:/ # cat proc/cpuload	cpu0	cpu1	cpu2	cpu3	cpu4	cpu5	cpu6	cpu7	cluster 1 avl performance pct	cluster 2 avl performance pct	cpu performance avl pct
614400 689000 768000 884000 1001000 1118000 1228800 1326000 1430000 1534000 1638000 1742000 1846000 1950000 2041000 online wfi idle util_pct totallime	1.94%	2.18%	2.91%	0.98%	1.42%	1.46%	2.48%	2.15%	2.16%	1.59%	2.02%
0.11%	0.12%	0.07%	0.03%	0.06%	0.09%	0.16%	0.05%	0.25%	0.13%	0.09%	0.07%
0.11%	0.11%	0.05%	0.08%	0.09%	0.03%	0.25%	0.03%	0.16%	0.13%	0.09%	0.07%
0.28%	0.23%	0.07%	0.08%	0.09%	0.06%	0.07%	0.03%	0.07%	0.07%	0.07%	0.07%
0.64%	0.53%	0.42%	0.14%	0.08%	0.06%	0.00%	0.00%	0.03%	0.00%	0.00%	0.00%
0.34%	0.14%	0.21%	0.08%	0.26%	0.06%	0.00%	0.06%	0.01%	0.06%	0.06%	0.06%
0.13%	0.11%	0.13%	0.09%	0.08%	0.06%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
0.17%	0.18%	0.23%	0.05%	0.14%	0.06%	0.03%	0.02%	0.02%	0.02%	0.02%	0.02%
0.27%	0.28%	0.09%	0.01%	0.05%	0.00%	0.02%	0.00%	0.00%	0.00%	0.00%	0.00%
0.11%	0.06%	0.01%	0.00%	0.01%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
0.07%	0.06%	0.06%	0.02%	0.00%	0.01%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
0.09%	0.02%	0.18%	0.00%	0.02%	0.01%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
0.08%	0.04%	0.02%	0.00%	0.02%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
0.03%	3.31%	0.02%	0.13%	0.08%	0.07%	0.07%	0.02%	0.05%	0.07%	0.02%	0.05%
5.71%	7.54%	4.73%	1.75%	2.43%	1.96%	3.43%	0.02%	0.71%	3.43%	0.02%	0.71%
3.69%	3.49%	1.01%	1.38%	1.49%	1.61%	3.22%	0.02%	0.84%	3.22%	0.02%	0.84%
94.28%	92.45%	95.26%	98.24%	97.56%	98.03%	96.56%	96.10%	96.10%	96.10%	96.10%	96.10%
3.27%	5.15%	2.05%	0.76%	0.93%	0.73%	1.17%	2.02%	2.02%	2.02%	2.02%	2.02%
15534840	15534864	15534864	15534875	15534885	15534895	15534908	15534916	15534916	15534916	15534916	15534916

2.1.2 **cpu_trans_table**

功能

扫描一段时间内 CPU 各个频点的跳变关系, 默认扫描 60s, 20ms 采样一次。

cpu_trans_table 输出的是二维矩阵, 展示了所有有关 CPU 频点跳变的细粒度信息。

工具运行时间按需设定, 单位: s (秒)。

使用方法

仅 user root/debug 版本支持, user 版本不支持。

```
adb shell
```

```
cd vendor/bin/power/cpu/trans_table
```

```
./cpu_trans_table -t 60
```

-t 后为扫描时间, 上述命令中 60 代表运行 60s。

运行结果

cpu_trans_table 运行 60s 后, 示例如下。

```
accumulate_time is 60
counts is 3000
cpu 8 cores.
```

	614400	768000	884000	1040000	1228800	1378000	1536000	1612000
614400	2667	24	13	6	6	4	6	17
768000	18	1	5	2	0	0	1	2
884000	15	1	0	0	0	1	1	2
1040000	7	0	0	0	0	0	1	2
1228800	3	2	0	1	1	0	0	1
1378000	4	1	0	0	1	0	0	0
1536000	9	0	0	0	0	0	1	2
1612000	21	0	2	1	0	1	1	145

	614400	768000	884000	1040000	1228800	1378000	1536000	1612000
614400	2773	16	4	0	0	7	2	7
768000	14	2	0	0	0	2	0	2
884000	6	1	2	0	0	0	0	0
1040000	6	0	0	1	0	0	1	3
1228800	2	0	0	0	1	0	0	0
1378000	5	1	2	0	1	1	0	1
1536000	0	0	1	1	1	0	0	1
1612000	12	0	0	0	0	1	1	109

输出结果中展示了每行、每列的实际频点（freq）值，其中，条目<i, j>（第 i 行，第 j 列）表示 freq_i 到 freq_j 的转换次数。

2.1.3 fix_cpu_freq

功能

固定 CPU 大小核频点。

fix_cpu_freq 运行过程中会提示“input fix freq”，按照提示输入固定的频点即可。输入的固定频点必须在可用频点中选取。

设备重启后失效，需重新设置。

使用方法

仅 user root/debug 版本支持，user 版本不支持。

```
adb shell
cd vendor/bin/power/cpu/fix_freq
./fix_cpu_freq
```

运行结果

运行 fix_cpu_freq，输入固定的 CPU 大小核频点后，示例如下。

```
s9863alh10:/vendor/bin/power/cpu/fix_freq # ./fix_cpu_freq
lit core
768000 884000 1000000 1100000 1200000
input fix freq
884000
cur_freq:884000

big core
768000 1050000 1225000 1400000 1500000 1600000
input fix freq
1050000
cur_freq:1050000
```

其中：

- lit core 下打印的是小核所有的可用频点。
- big core 下打印的是大核所有的可用频点。
- cur_freq 后显示的是 CPU 的当前频点。
cur_freq 显示的频点和输入的频点一致，代表固定成功。

上图表示：CPU 小核固定在 884000Hz，大核固定在 1050000Hz。

除 fix_cpu_freq 工具外，还可以使用命令固定 CPU 大小核频点，以固定 CPU0 频点为例，命令如下。

```
echo userspace > /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
echo xxx > /sys/devices/system/cpu/cpu0/cpufreq/scaling_setspeed
```

其中，xxx 为固定的频点，单位 Hz。

2.1.4 lit_cpu_freq

功能

限制 CPU 大小核最大频点。

lit_cpu_freq 运行过程中会提示 “input lit max freq”，按照提示输入限定的最大频点即可。输入的最大频点必须在可用频点中选取。

设备重启后失效，需重新设置。

使用方法

仅 user root/debug 版本支持，user 版本不支持。

```
adb shell
cd vendor/bin/power/cpu/lit_freq
./lit_cpu_freq
```


运行结果

运行 `lix_cpu_freq`，输入 CPU 大小核的最大频点后，示例如下。

```
lit core
614400 768000 884000 1040000 1228800 1378000 1536000 1612000
input lit max freq
1612000
Max_freq:1612000

big core
768000 1040000 1228800 1404000 1536000 1612000
input lit max freq
1404000
Max_freq:1612000
```

其中：

- `lit core` 下打印的是小核所有的可用频点。
- `big core` 下打印的是大核所有的可用频点。
- `Max_freq` 后显示的是 CPU 的最大频点。

可通过 `cpu_loading` 工具打印的频点利用率来确认最大频点是否限定成功，如限定成功，高于限定频点的频点利用率为 0。

除 `lix_cpu_freq` 工具外，还可以使用命令限定 CPU 大小核最大频点，以限定 CPU0 的最大频点为例，命令如下。

```
echo xxx> /sys/devices/system/cpu/cpu0/cpufreq/scaling_max_freq
```

其中，xxx 为限定的最大频点，单位 Hz。

2.2 GPU

GPU 相关的功耗工具仅 debug 版本支持。

2.2.1 gpu_loading

功能

统计一段时间内 GPU 各个频点的占比和使用情况。

工具运行时间按需设定，单位：s（秒）。

使用方法

```
adb shell
cd vendor/bin/power/gpu/loading
./gpu_loading -t 60
```


-t 后为运行时间，上述命令中 60 代表运行 60s。

运行结果

gpu_loading 运行 60s 后，示例如下。

图2-4 SC9863A 项目 gpu_loading 运行结果

```
accumulate_time is 60(s)
256: 58.08%      utilisation:28.88%
384: 30.92%      utilisation:21.09%
550: 11.00%      utilisation:23.70%
Averag GPU Utilisation : 25.90%
```

图 2-4 输出结果表示 60s 内：

- GPU 运行在 256MHz、384MHz、550MHz 频点上的占比分别为 58.08%、30.92%、11.00%。
- 频点 256MHz、384MHz、550MHz 的利用率依次为 28.88%、21.09%、23.7%。
- GPU 平均利用率为 25.90%。

图2-5 UIS7865 项目 gpu_loading 运行结果

```
*****gpu_start_testing*****
GPUINFO      0      384000000      512000000      614400000      650000000
FreqTime     0.00%      6.19%      0.49%      0.00%      93.32%
FreqLoading  0.00%      1.28%      0.15%      0.00%      17.94%
```

其中：

- GPUINFO：频点信息。
- FreqTime：GPU 每个频点运行时间对应总时间的占比。
- FreqLoading：频点利用率。

图 2-5 输出结果表示：

- GPU 运行在 384MHz、512MHz、614.4MHz、650MHz 频点上的占比分别为 6.19%、0.49%、0.00%、93.32%。
- 频点 384MHz、512MHz、614.4MHz、650MHz 的利用率依次为 1.28%、0.15%、0.00%、17.94%。

2.2.2 gpu_trans_table

功能

扫描一段时间内 GPU 各个频点的跳变关系。

gpu_trans_table 输出的是二维矩阵，展示了所有有关 GPU 频点跳变的细粒度信息。

工具运行时间按需设定，单位：s（秒）。

使用方法

```
adb shell
cd vendor/bin/power/gpu/trans_table
./gpu_trans_table -t 60
```

-t 后为扫描时间，上述命令中 60 代表运行 60s。

运行结果

gpu_trans_table 运行 60s 后，示例如下。

```
accumulate_time is 60
counts is 3000
650      650      614      512      384
650      0         0         0         0
614      0      2999      0         0
512      0         0         0         0
384      0         0         0         0
```

输出结果中展示了每行、每列的实际频点（freq）值。其中，条目<i, j>（第 i 行，第 j 列）表示 freq_i 到 freq_j 的转换次数。

2.2.3 fix_gpu_freq

功能

固定 GPU 频点。

fix_gpu_freq 运行过程中会提示“input fix freq”，按照提示输入固定的频点即可。输入的固定频点必须在可用频点中选取。

设备重启后失效，需重新设置。

使用方法

```
adb shell
cd vendor/bin/power/gpu/fix_freq
./fix_gpu_freq
```

运行结果

运行 fix_gpu_freq，输入固定的 GPU 频点后，结果如下。

```
gpu available freq:
650000000      614400000      512000000      384000000
input fix freq
614400000
cur_freq:614400000
```

cur_freq 显示的频点和输入的频点一致，代表固定成功。

上图输出结果表示将 GPU 的频点固定在 614.4MHz。

此外，可以通过命令限制 GPU 的最大频点，命令如下。

```
echo xxx >/sys/devices/platform/soc/soc:mm/600000000.gpu/devfreq/600000000.gpu/max_freq
```

其中，xxx 为限定的最大频点，单位 Hz。

2.3 DDR

2.3.1 ddr_loading

功能

统计一段时间内 DDR 各个频点运行时间的占比。

工具运行时间按需设定，单位：s（秒）。

使用方法

- user 版本

```
adb shell cp /vendor/bin/power/ddr/loading/ddr_loading data/local/tmp/  
adb shell ./data/local/tmp/ddr_loading -t 60
```

- user root/debug 版本

```
adb shell  
cd vendor/bin/power/ddr/loading  
./ddr_loading -t 60
```

-t 后为运行时间，上述命令中 60 代表运行 60s。

运行结果

ddr_loading 运行 60s 后，示例如下。

图2-6 SC9863A 项目 ddr_loading 运行结果

```
s9863a1h10:/vendor/bin/power/ddr/Loading # ./ddr_loading -t 60
./ddr_loading -t 60
accumulate_time is 60(s)
time_use is 55463.82ms
160:    0.00%
233:    0.00%
311:   98.84%
400:    0.92%
533:    0.00%
622:    0.20%
800:    0.03%
933:    0.00%
unit:10ms
theory_bw = 1280000    1864000 2488000 3200000 4264000 4976000 6400000 7464000
overflow   = 448000 652400 870800 1120000 1492400 1741600 2240000 4294967295
underflow  = 0 348000 552400 770800 1020000 1020000 1641600 2140000
```

图 2-6 输出结果表示 60s 内，DDR 运行在 160MHz、233MHz、311MHz、400MHz、533MHz、622MHz、800MHz、933MHz 的占比依次为 0.00%、0.00%、98.84%、0.92%、0.00%、0.20%、0.03%、0.00%。

图2-7 UIS7865 项目 ddr_loading 运行结果

```
uis7865_6h10:/ # cd vendor/bin/power/ddr/loading
uis7865_6h10:/vendor/bin/power/ddr/loading # ./ddr_loading -t 60

DDRFreq      256Mhz  384Mhz  512Mhz  768Mhz  1024Mhz 1333Mhz 1600Mhz 0Mhz
FreqRatio    0.00%   0.00%   0.00%   99.94%  0.05%   0.01%   0.00%   0.00%

DDRState      idle time:      write_time:      read_time:      sref_time:      light_time
StateRatio    9.86%          0.17%           8.09%           43.18%          38.69%
```

图 2-7 输出结果表示 60s 内，DDR 运行在 256MHz、384MHz、512MHz、768MHz、1024MHz、1333MHz、1600MHz 的占比依次为 0.00%、0.00%、0.00%、99.94%、0.05%、0.01%、0.00%。

DDR 运行在高频的占比越高，功耗越高。

2.3.2 ddr_trans_table

功能

扫描一段时间内 DDR 各个频点的跳变关系。

ddr_trans_table 输出的是二维矩阵，展示了所有有关 DDR 频点的跳变的细粒度信息。

工具运行时间按需设定，单位：s（秒）

使用方法

仅 user root/debug 版本支持，user 版本不支持。

```
adb shell
cd vendor/bin/power/ddr/trans_table
```

```
./ddr_trans_table -t 60
```

-t 后为扫描时间，上述命令中 60 代表运行 60s。

运行结果

ddr_trans_table 运行 60s 后，示例如下。

accumulate_time	is	60					
256	384	512	768	1024	1333	1600	
256	384	4	1	0	0	0	0
384	5	1	0	0	0	0	0
512	0	0	0	1	0	0	0
768	0	0	0	5372	62	19	0
1024	0	0	0	59	9	9	0
1333	0	0	0	22	6	45	0
1600	0	0	0	0	0	0	0

输出结果中展示了每行、每列的实际频点（freq）值，其中，条目<i, j>（第 i 行，第 j 列）表示 freq_i 到 freq_j 的转换次数。

2.3.3 fix_ddr_freq

功能

固定 DDR 频点。

fix_ddr_freq 运行过程中会提示“input fix freq”，按照提示输入固定的频点即可。输入的固定频点必须在可用频点中选取。

设备重启后失效，需重新设置。

使用方法

仅 user root/debug 版本支持，user 版本不支持。

```
adb shell
cd vendor/bin/power/ddr/fix_freq
./fix_ddr_freq
```

运行结果

运行 fix_ddr_freq，输入固定的 DDR 频点后，结果如下。

```
ddr available freq:
256    384    512    768    1024    1333    1600
input fix freq
384
cur_freq:384
```


cur_freq 显示的频点和输入的频点一致，代表固定成功。

上图输出结果表示将 DDR 频点固定在 384MHz。

除 fix_ddr_freq 工具外，还可以使用命令固定 DDR 频点，命令如下。

```
echo 0 > /sys/class/devfreq/scene-frequency/sprd_governor/auto_dfs_on_off  
echo xxx > /sys/class/devfreq/scene-frequency/sprd_governor/scaling_force_ddr_freq
```

其中，xxx 为固定的频点，单位 Hz。

2.3.4 ddr_bm

功能

抓取一段时间内 DDR 的带宽信息。

工具运行到指定时间后，会在 data/local/tmp 或 vendor/bin/power/ddr/bm 路径下生成带宽信息文件 xxx_ddr_bm.csv，具体路径请根据项目实际情况确认。

每次抓取带宽信息前，需重启设备。

工具运行时间按需设定，单位：s（秒）。

使用方法

仅 Debug 版本支持。

```
adb shell  
cd vendor/bin/power/ddr/bm  
./tool/ddr_bm -t 240
```

-t 后为测试时间，上述命令中 240 代表运行 240s。

注意

Android 12 及更高版本移除了路径/vendor/bin/power/ddr/bm/tool 下的解析工具 ddr_bm_log，因此，执行 ddr_bm 脚本前，需先手动将 ddr_bm_log 解析工具 push 到/vendor/bin/power/ddr/bm/tool 下。

如需 ddr_bm_log 工具，请联系 CPM 获取。

运行结果

ddr_bm 运行 240s 后，结果如下。

图2-8 SC9863A 项目 ddr_bm 运行结果

```
s9863a1h10:/vendor/bin/power/ddr/bm # ./tool/ddr_bm -t 240  
./ddr_bm -t 240  
accumulate_time is 240(s)  
s9863a1h10:/vendor/bin/power/ddr/bm # ls  
ls  
tool readme shark13_ddr_bm.csv
```

图2-9 UIS7865 项目 ddr_bm 运行结果

```
C:\Users\>adb root

C:\Users\>adb remount
Verity is already disabled
Remounted /system as RW
Remounted /system_ext as RW
Remounted /vendor as RW
Remounted /odm as RW
Remounted /product as RW
Remounted /vendor_dlkm as RW
Remounted /system_dlkm as RW
Remount succeeded

C:\Users\>adb push D:\powertools\ddr_bm_log\vendor/bin/power/ddr/bm/tool
D:\powertools\ddr_bm_log: 1 file pushed. 3.5 MB/s (641298 bytes in 0.173s)

C:\Users\>adb shell
uis7865_6h10:/ # ./vendor/bin/power/ddr/bm/tool/ddr_bm -t 30
start set_ddr_mode

C:\Users\>adb shell
uis7865_6h10:/ # cd data/local/tmp/
uis7865_6h10:/data/local/tmp # ls
qogirl6_ddr_bm.csv
```

2.4 其他

2.4.1 loading

功能

同时统计一段时间内 CPU、GPU、DDR 的 loading 信息。

工具运行时间按需设定，单位：s（秒）。

使用方法

CPU、DDR 信息 user 及 debug 版本可获取，GPU 信息 debug 版本才能获取。

```
adb shell
cd vendor/bin/power/loadings/tool
sh loading.sh 10 10 10 //10、10、10分别表示CPU、GPU、DDR loading的运行时间。
```

运行结果

loading 运行 10s 后，示例如下。

```

cpu_loading:
testing 10s

cpuN      614400  768000  884000  1040000  1228800  1378000  1536000  1612000  online  idle
cpu0      2.79%   0.01%   0.00%   0.00%   0.01%   0.00%   0.00%   0.09%   2.90%  97.10%
cpu1      0.29%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.01%   0.30%  99.70%
cpu2      0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%  100.00%
cpu3      0.58%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.02%   0.60%  99.40%
cpu4      0.19%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.01%   0.20%  99.80%
cpu5      0.10%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.10%  99.90%
Freq      96.20%  0.20%   0.00%   0.10%   0.30%   0.00%   0.00%   3.20%
sum:      26630.21kHz
power:    0.00

cpuN      768000  1040000  1228800  1404000  1536000  1612000  online  idle
cpu6      0.10%   0.00%   0.00%   0.00%   0.00%   0.00%   0.10%  99.90%
cpu7      0.00%   0.00%   0.00%   0.00%   0.00%   0.00%   0.00%  100.00%
Freq      100.00% 0.00%   0.00%   0.00%   0.00%   0.00%
sum:      767.23kHz
power:    0.00

sum_total: 27397.44KHZ
power_total: 0.00

gpu_loading:
*****gpu_start_testing*****

Used Old GPU Data Sampling Way?
polling interval is 50(ms)
counts is 200
cat: /d/mali0/gpu_utilisation: No such file or directory
gpu_utilisation does not exist
650: 0          614: 0          512: 0          384: 1
384: 0.50%      utili:0.00%
Averag GPU Utili:0.00%

ddr_loading:

DDRFreq      256Mhz  384Mhz  512Mhz  768Mhz  1024Mhz  1333Mhz  1600Mhz  0Mhz
FreqRatio    0.00%   0.00%   0.00%   58.40%  7.36%   34.24%   0.00%   0.00%

DDRState      idle_time:  write_time:  read_time:  sref_time:  light_time:
StateRatio    19.58%    21.77%     42.66%     9.25%      6.75%
    
```

2.4.2 brightness

功能

查看背光节点信息。

背光节点的取值范围为[0,255]。通常情况下，背光节点值越高，亮度也越高，功耗也会越大。

使用方法

```

adb shell
cd vendor/bin/power/backlight
./brightness
    
```

运行结果

运行 brightness 脚本，示例如下。

```
s9863a1h10:/vendor/bin/power/backlight # ./brightness
./brightness
brightness: 0
brightness: 0
brightness: 0
brightness: 92
brightness: 92
brightness: 92
brightness: 92
brightness: 170
brightness: 204
brightness: 204
brightness: 204
brightness: 204
```

除可通过 brightness 工具查看背光节点信息外，还可以使用命令设定/读取背光节点信息，命令如下。

- 设定背光值

```
adb shell settings put system screen_brightness xxx //xxx为背光值
```

- 读取背光值

```
adb shell settings get system screen_brightness
```

2.4.3 top

功能

实时抓取系统中各个进程的资源占用状况。

该命令可以按 CPU 使用率、内存使用和执行时间对进程进行排序。

使用方法

top 常用的参数如下。

- [-m]: 最多显示的进程数。
- [-n]: top 进程刷新次数。需要设置固定的刷新次数，可以设置该项。
- [-d]: 实时刷新的时间间隔，单位 s，默认 5s 刷新一次。
- [-s]: 排序的类型，默认按 CPU 使用率排序，一般无需修改。
- [-p]: 指定显示某个 PID 的相关信息。

例如，显示 10 个进程的资源占用情况，示例如下。

```
adb shell top -m 10
```

top 各个参数可组合使用。

例如，当定位到某个异常进程后，可使用命令“adb shell top -H -p [pid]”进一步查看该进程内部的线程运行情况，可以用于对比分析版本升级间的线程。

运行结果

显示 10 个进程的资源占用情况，运行结果如下。

```
Tasks: 776 total, 1 running, 775 sleeping, 0 stopped, 0 zombie
Mem: 2804736K total, 2734360K used, 70376K free, 3192K buffers
Swap: 1738932K total, 768976K used, 969956K free, 1303036K cached
800%cpu 6%user 2%nice 5%sys 782%idle 3%iow 1%irq 0%sig 0%host
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	ARGS
7255	u0_a120	20	0	17G	98M	68M	S	5.0	3.5	0:29.39	com.android.vending
972	system	18	-2	22G	246M	176M	S	3.3	8.9	2:30.28	system_server
9218	root	20	0	10G	5.4M	3.7M	R	2.3	0.1	0:01.84	top -m 10
8695	root	20	0	0	0	0	I	0.6	0.0	0:00.30	[kworker/3:0-mm_percpu_wq]
7455	u0_a120	20	0	16G	54M	46M	S	0.6	1.9	0:01.59	com.android.vending:background
578	system	20	0	10G	2.3M	2.2M	S	0.6	0.0	0:01.15	vendor.sprd.hardware.thermal@2.0-service
2042	u0_a121	20	0	17G	104M	87M	S	0.3	3.7	0:34.72	com.google.android.gms
789	root	20	0	10G	2.8M	2.8M	S	0.3	0.1	0:00.23	charged
305	root	RT	0	0	0	0	S	0.3	0.0	0:02.20	[usg_sugov:0]
285	system	20	0	10G	3.3M	2.6M	S	0.3	0.1	0:04.17	servicemanager

其中：

- [%CPU]：各个进程对 CPU 的使用率。
- ARGS：进程名称。